

Unit 4: Incident Post-Mortem Framework

Case Study: British Airways Magecart Attack (2018)

In 2018, British Airways (BA) suffered a major data breach affecting approximately 500,000 customers. Attackers injected malicious code into the airline's website and mobile app, capturing personal and financial information directly as customers entered it. This incident demonstrates how third-party vulnerabilities and insufficient monitoring can be exploited, even without direct server access.

1. Threat Actor Profiling

- **Type:** Cybercriminals using Magecart-style digital skimming.
- **Motivation:** Financial gain from stolen payment card data.
- **Skills:** Moderate web exploitation knowledge, specifically client-side attacks.
- **Behaviour:** Exploited third-party scripts without detection for approximately two weeks.
- **Impact:** Accessed sensitive customer information, potential reputational and financial loss.

Even skilled attackers can succeed when client-side protections and third-party governance are weak.

2. Historical Incident Overview

Website and Mobile App Breach

- Attackers injected malicious JavaScript into the modernizr.js library on payment pages.
- Captured customers' personal and payment details.
- Breach lasted **around two weeks** before discovery by a third-party researcher.
- BA temporarily took the website offline to investigate and resolve the issue.

3. Tactics, Techniques, and Procedures (TTPs)

- Exploited **third-party JavaScript library** on payment pages.
- Deployed **digital skimming code** to capture sensitive data in real-time.
- Avoided detection due to **lack of client-side monitoring** and no Subresource Integrity (SRI) checks.
- Operated discreetly to remain undetected for multiple days.

4. Indicators of Compromise (IoCs)

Technical Indicators

- Malicious code embedded in modernizr.js.
- Skimming scripts intercepting payment forms.
- Unusual outbound connections from client-side scripts.

Behavioural Indicators

- Breach activity over several days without server alerts.
- Third-party researchers discovered anomaly, not internal monitoring.
- Customers reporting fraudulent transactions after online payments.

5. Vulnerability and Targeting Analysis

Exploited Vulnerabilities

- Inadequate controls over **third-party scripts**.
- Lack of **Subresource Integrity (SRI)** to prevent execution of modified scripts.
- No **client-side monitoring** or alerting on live page changes.

Target Selection

- High visibility: BA's online payment systems handle thousands of transactions daily.
- Weak client-side defences: Focused on scripts rather than server security.
- Multiple entry points: Website and mobile app payment forms allowed access to sensitive data.
- Financial impact: Stolen payment card information used for fraudulent transactions.

6. Key Lessons and Mitigation Strategies

- **Third-Party Script Management:** Implement strict approval, monitoring, and change control for all external scripts.
- **Subresource Integrity (SRI):** Use SRI hashes to prevent altered scripts from executing.
- **Client-Side Monitoring:** Deploy solutions to monitor DOM and network traffic for malicious activity.
- **Incident Response:** Improve detection capabilities and reduce reliance on third-party discovery.
- **Staff Awareness:** Educate developers and IT teams on client-side security risks and governance.

Conclusion

The British Airways Magecart attack highlights the risks associated with third-party scripts and insufficient client-side monitoring. Even sophisticated attackers can exploit small vulnerabilities to access sensitive data. Organisations must enforce robust controls over all external code, monitor client-side activity, and maintain a proactive security culture to prevent similar breaches.

Coding Perspective: What British Airways Did Wrong

British Airways included a script on their payment page like this

Example:

```
<script src="https://some-third-party-cdn.com/track.js"></script>
```

Fixed version" approach (most common & safest)

Websites almost always link to a **specific version** of the script, not a file that changes.

Example:

```
<script
  src="https://cdn.jsdelivr.net/npm/jquery@3.6.3/dist/jquery.min.js"
  integrity="sha384-ABC123..."
  crossorigin="anonymous">
</script>
```

- This version (3.6.3) **never changes** on the CDN.
- So **the SRI hash stays valid forever**.
- No need for contact or updates.

This is how 99% of production websites do it.

Because BA did not follow above. This means:

- Every time a user visited the BA website,
- The browser fetched **the latest live file** from the CDN,
- Without **any verification, approval, or review** from BA.

So if the file on the CDN changed...

- BA would automatically load the changed file
- BA wouldn't know
- The browser wouldn't know
- There was **no hash, no integrity check, no version pinning**

This is exactly why the attackers succeeded.

BA was automatically used whatever the CDN served.

This is the core design mistake.

They essentially trusted the CDN blindly.

What the attackers did

Magecart didn't hack BA's servers — they hacked the **third-party CDN** or the account that managed the file.

Then they added a malicious line at the bottom of the JavaScript file:

```
// original library code...  
// malicious addition  
sendCardDataToAttacker(cardNumber, expiry, cvv);
```

Because BA hadn't pinned a version or used SRI:

- ⚠ BA's site loaded the modified script automatically
- ⚠ No integrity check was triggered
- ⚠ Users' card data was skimmed silently
- ⚠ BA didn't detect it for ~2 weeks

The attack succeeded **without touching BA's servers**.